郑州工商学院
Zhengzhou Technology and Business University

# 本科毕业设计

## 基于深度学习的中文垃圾短信识别系统设计与实现

| | |
|---|---|
| 院部名称 | 信息工程学院 |
| 姓　　名 | 潘宇航 |
| 学　　号 | 220529090143 |
| 专　　业 | 数据科学与大数据技术 |
| 届　　别 | 2026 届 |
| 指导教师 | |

2026 年 4 月 20 日

# 基于深度学习的中文垃圾短信识别系统设计与实现

**摘要：**随着移动通信和互联网服务的发展，短信仍然被广泛用于通知、验证和业务提醒，但广告推广、诈骗诱导、虚假中奖等垃圾短信也随之增加。中文垃圾短信具有文本短、表达灵活、口语化明显和类别不均衡等特点，单纯依靠关键词规则容易出现漏判和误判。本文围绕中文垃圾短信识别任务，设计并实现了一套基于深度学习的识别系统。研究以约 80 万条中文短信数据为基础，完成文本清洗、中文分词、停用词过滤、同义词增强、分层采样和类别权重设置等预处理工作，并分别构建 BERT 和 LSTM 两种模型进行对比。实验结果表明，BERT 和 LSTM 均可完成短信识别任务，其中 BERT 在中文短文本语义建模和系统部署稳定性方面更适合作为主要识别模型。系统采用 Django 作为后端框架，Vue 作为前端框架，实现了短信单条识别、批量识别、模型训练、模型对比和识别历史管理等功能。测试结果表明，系统能够较稳定地完成中文短信分类任务，具有一定的实用价值。

**关键词：**深度学习；BERT 模型；LSTM 模型；自然语言处理；垃圾短信

识别

# Design and Implementation of a Chinese Spam SMS Recognition System Based on Deep Learning

**Abstract:** With the development of mobile communication and Internet services, SMS is still widely used for notifications, verification, and service reminders. However, spam messages such as advertisements, fraud messages, and false prize notices have also increased. Chinese spam SMS is short, flexible, colloquial, and imbalanced in category distribution, which makes rule-based methods prone to false detection and missed detection. This thesis designs and implements a Chinese spam SMS recognition system based on deep learning. Based on about 800,000 Chinese SMS records, text cleaning, Chinese word segmentation, stop-word filtering, synonym augmentation, stratified sampling, and class weighting are performed. BERT and LSTM models are constructed and compared. The experimental results show that both BERT and LSTM can complete spam SMS recognition, while BERT is more suitable as the main model in terms of Chinese short-text semantic modeling and deployment stability. The system uses Django as the backend framework and Vue as the frontend framework, implementing single-message recognition, batch recognition, model training, model comparison, and recognition history management. The test results show that the system can complete Chinese SMS classification tasks stably and has practical value.

**Keywords:** deep learning; BERT model; LSTM model; natural language processing; spam SMS recognition

# 目录

1■□□.....1

1.1■□□□□.....1

1.1.1■□□□□.....1

1.1.2■□□□□.....1

1.2■□□□□.....1

1.3■□□□□.....1

1.3.1■□□□□.....1

1.3.2■□□□□.....2

1.4■□□□□.....2

1.4.1■□□□□.....2

1.4.2■□□□□.....2

1.4.3■□□□□.....3

2■□□.....4

2.1■□□□□.....4

2.1.1■□□□□.....4

2.1.2■□□□□.....4

2.1.3■□□□□.....4

2.2■□□□□.....4

2.2.1■□□□□.....4

2.2.2■□□□□.....5

3■□□.....6

3.1■□□□□.....6

3.2■□□□□.....6

3.3■□□□□.....6

3.3.1■□□□□.....6

3.3.2■□□□□.....6

4■□□.....8

4.1■□□□□.....8

4.1.1■□□□□.....8

4.2■□□□□.....8

4.2.1■□□□□.....8

4.2.2■□□□□.....9

结论.....11

参考文献.....12

致谢.....13

附录 A 程序源代码.....14

# 1 绪论

## 1.1 研究背景

随着移动互联网和在线服务的持续普及，短信仍然是验证码、业务通知、营销触达和公共服务提醒的重要渠道。与社交软件消息相比，短信具有到达率高、使用门槛低、对终端设备要求少等特点，因此在金融、运营商、电商、政务通知等场景中仍被广泛使用。与此同时，广告推广、诱导链接、虚假中奖、贷款诈骗等垃圾短信也长期存在，影响用户通信体验，并可能带来个人信息泄露和财产损失风险。

传统垃圾短信过滤主要依赖黑名单、关键词匹配和人工规则。该类方法实现简单、可解释性较强，但面对中文短文本中的谐音替换、符号拆分、口语化表达和变体诱导话术时，规则维护成本较高，且容易出现漏判。垃圾短信的文本长度通常较短，部分短信还会混合数字、英文、表情符号和非常规空格，单纯依靠人工特征很难完整覆盖其语义特征。

深度学习为垃圾短信识别提供了新的技术路径。LSTM 能够利用门控机制建模文本序列关系，适合从分词序列中提取上下文依赖；BERT 等预训练语言

模型能够基于 Transformer 结构学习双向语义表示，对短文本中的隐含语义和上下文关系具有更强表达能力。因此，结合 BERT 和 LSTM 开展中文垃圾短信识别实验，并将模型嵌入实际 Web 系统，能够使研究同时具有算法分析价值和系统应用价值。

## 1.2 研究目的和意义

### 1.2.1 研究目的

本文的研究目的是设计并实现一套基于深度学习的中文垃圾短信识别系统。系统围绕中文短信二分类任务展开，完成数据预处理、模型训练、模型评价、短信识别和识别历史管理等环节，使用户能够在浏览器中输入短信文本并获得正常短信或垃圾短信的判断结果。

在模型层面，本文选择 BERT 和 LSTM 作为主要对比对象。BERT 用于验证预训练语言模型在中文短文本语义建模中的优势，LSTM 用于验证轻量序列模型在分词序列上的识别能力。通过准确率、精确率、召回率、F1 值、验证损失和混淆矩阵等指标，对两类模型进行对比分析，并说明系统最终模型选用依据。

### 1.2.2 研究意义

从理论意义看，本文将自然语言处理、深度学习和文本分类评价方法结合起来，梳理了中文垃圾短信识别从文本清洗、分词编码到模型训练的完整流程。通过对 BERT 与 LSTM 的实验对比，可以观察不同模型结构在短文本二分类任务中的表现差异。

从应用意义看，垃圾短信识别系统能够辅助用户快速判断短信风险，减少无效营销和诈骗短信对正常通信的干扰。系统采用 Django 和 Vue 实现前后端分离，能够把模型训练结果、识别接口和历史记录管理连接起来，展示深度学习模型从实验到应用落地的过程。

## 1.3 国内外文献综述

### 1.3.1 国外研究现状

国外关于垃圾短信和垃圾邮件过滤的研究起步较早，早期方法多采用朴素贝叶斯、支持向量机、K 近邻和 TF-IDF 特征组合等传统机器学习方法。随着深度学习的发展，CNN、RNN、LSTM 以及 Transformer 结构逐渐应用于垃圾消息检测，研究重点从人工关键词和浅层统计特征转向语义表示学习。

近年来，部分研究开始将 BERT 嵌入、图卷积网络、多通道特征融合和不平衡学习方法用于 SMS spam detection 任务。这类方法能够更好地处理短信文

本短、噪声多、语义隐藏和样本分布不均衡等问题。国外研究通常同时使用准确率、召回率、F1 值、AUC、平均精度和混淆矩阵评价模型效果，其中召回率在垃圾短信检测中尤为重要，因为漏判垃圾短信可能带来实际安全风险。

### 1.3.2 国内研究现状

国内研究主要围绕中文文本特点展开。由于中文没有天然空格分隔，分词质量会直接影响传统序列模型和词向量模型的输入效果。早期研究常采用 jieba 分词、停用词过滤、TF-IDF 和传统分类器完成识别；近年来，BERT、BiLSTM、CNN-LSTM 等模型逐渐被用于中文垃圾短信识别和中文短文本分类。

已有研究表明，BERT 模型能够利用双向上下文语义表示提高中文短文本分类效果，LSTM 及其双向结构则在序列建模和轻量部署方面具有一定优势。现有研究不足之处在于，部分工作停留在算法验证阶段，缺少与 Web 系统结合的实现过程；也有部分系统论文只展示页面，未充分说明训练曲线、损失函数、测试集评价和模型选择依据。本文针对这些问题，将模型实验、系统实现和测试评价统一组织。

### 1.3.3 研究评述

综合国内外研究可以看出，中文垃圾短信识别任务既需要自然语言处理方

法，也需要工程化系统实现能力。模型选择不能只看单一准确率，还应结合召回率、F1 值、损失曲线、推理速度和系统部署方式进行综合判断。本文在前人研究基础上，构建了 BERT 与 LSTM 对比实验，并围绕 Django 后端、Vue 前端和 SQLite 数据存储完成系统实现。

## 1.4 研究的主要内容和方法

### 1.4.1 研究内容

本文主要研究内容包括四个方面。第一，构建中文短信数据处理流程，对原始 CSV 数据进行字段读取、文本清洗、中文分词、停用词过滤、同义词增强和编码转换；第二，设计 BERT 和 LSTM 两类识别模型，分别说明其输入格式、网络结构和训练过程；第三，结合训练曲线、混淆矩阵和测试集三线表评价模型效果；第四，基于 Django 和 Vue 实现中文垃圾短信识别系统，完成短信分类、模型训练、模型对比和识别历史管理等功能。

### 1.4.2 研究方法

本文采用文献研究法、实验对比法和系统实现法开展研究。文献研究法用于梳理垃圾短信识别、中文短文本分类和预训练语言模型的相关成果；实验对比法用于在相同数据处理流程下比较 BERT 和 LSTM 模型；系统实现法用于验

证模型在 Web 系统中的调用效果，使论文内容与实际代码结构保持一致。

# 2 相关理论与技术介绍

## 2.1 深度学习基础

深度学习是机器学习的重要分支，其核心思想是通过多层神经网络从数据中自动学习特征表示。对于文本分类任务而言，深度学习模型能够把词语、短语和句子映射为向量表示，并通过训练过程不断调整参数，使模型能够根据文本语义输出分类结果。

中文垃圾短信识别属于典型的短文本二分类问题。短信文本长度短、噪声多、表达灵活，常见垃圾短信会使用优惠、中奖、贷款、验证码、账户异常等诱导性词汇，也会通过符号、空格和谐音规避规则检测。深度学习模型能够减少人工特征设计依赖，在一定程度上提高对变体表达的适应能力。

本文重点使用 BERT 和 LSTM 两类模型。BERT 偏重上下文语义理解，适合处理复杂语义和短文本隐含含义；LSTM 偏重序列关系建模，结构相对轻量，训练速度较快。两种模型的对比能够体现预训练语言模型和传统循环神经网络在中文短信分类任务中的差异。

## 2.2 自然语言处理技术

自然语言处理是使计算机理解、分析和生成自然语言的重要技术。在本系

统中，自然语言处理主要用于将原始中文短信转换为模型可以接收的数值输入。处理过程包括文本清洗、中文分词、停用词过滤、同义词增强、Token 编码和序列补齐等步骤。

对于 LSTM 模型，系统采用 jieba 分词将短信拆分为词语序列，并结合停用词表删除“的”“了”“是”等低信息量词语。分词后的文本通过词表映射为整数序列，再经过 padding 或截断保持固定长度，最后输入 Embedding 层和 BiLSTM 层。对于 BERT 模型，系统使用 BERT Tokenizer 完成子词切分，生成 input_ids 和 attention_mask，使模型能够区分有效文本与补齐部分。

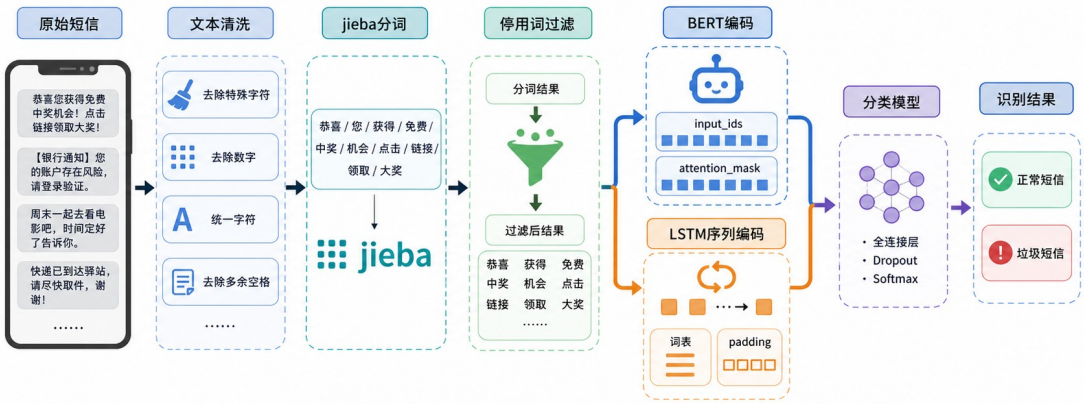


图 2-1 中文垃圾短信识别的自然语言处理流程

## 2.3 PyTorch 框架

PyTorch 是深度学习研究和工程开发中常用的框架，具有动态图机制、模块

化网络定义方式和灵活的张量计算能力。本文使用 PyTorch 定义 LSTM 分类模型，并通过 transformers 库加载 BERT 预训练模型。

在训练过程中，系统使用 DataLoader 按批次读取短信样本，使用 AdamW 优化器更新模型参数，并通过损失函数变化观察模型收敛状态。BERT 模型通过 BertForSequenceClassification 加载，LSTM 模型则由 Embedding 层、双向 LSTM 层、Dropout 层和全连接分类层组成。两种模型均输出垃圾短信类别的概率，并按阈值转换为最终标签。

## 2.4 Django 框架

Django 是基于 Python 语言的 Web 开发框架，提供 URL 路由、视图函数、ORM 数据库操作、中间件和后台管理等功能。本文后端使用 Django 封装数据处理、模型训练、模型评估、短信预测和历史记录查询接口，使前端能够通过 HTTP 请求调用模型能力。

系统后端通过 Django ORM 定义模型信息、模型评价结果和识别记录等数据表，训练完成后的指标、模型路径、训练历史和用户识别记录均可持久化保存。Django REST 接口与前端 Vue 页面配合，实现前后端分离的系统结构。

## 2.5 Vue 框架

Vue 是常用的前端 JavaScript 框架，具有组件化、响应式数据绑定和生态完善等特点。本文前端使用 Vue 构建系统主页面、短信分类识别页面、模型训练页面和识别历史页面，并通过 Axios 与 Django 后端接口交互。

在页面实现中，Vue 负责表单输入、按钮交互、训练进度展示、分类结果展示和历史记录表格渲染。前端不直接执行模型推理，而是将短信内容、模型类型和训练参数提交给后端，由后端完成预处理、模型加载和结果计算。

# 3 数据集介绍及处理

## 3.1 数据集介绍

本文使用的中文垃圾短信数据集以 CSV 文件形式保存，共包含 800000 条短信样本。每条记录由样本编号、类别标签和短信文本组成，其中标签 0 表示正常短信，标签 1 表示垃圾短信。经统计，正常短信共有 720000 条，占总样本的 90.0%；垃圾短信共有 80000 条，占总样本的 10.0%。数据集类别比例约为 9:1，存在明显类别不均衡现象。

正常短信主要包括新闻片段、生活交流、通知提醒和普通文本；垃圾短信主要包括营销推广、培训招生、优惠诱导、理财贷款、联系方式引流等内容。由于中文短信长度较短，部分垃圾短信会通过特殊符号、空格、表情符号或英文字符干扰规则匹配，因此需要在模型训练前进行规范化处理。

表 3-1 中文垃圾短信数据集统计

| 类别 | 标签 | 样本数量 | 占比 | 说明 |
|---|---|---|---|---|
| 正常短信 | 0 | 720000 | 90.0% | 普通通知、交流文本、新闻片段等 |
| 垃圾短信 | 1 | 80000 | 10.0% | 广告推广、诈骗诱导、培训招生等 |
| 合计 | - | 800000 | 100% | 用于模型训练、验证和系统测试 |

图 3-1 中文垃圾短信数据集类别分布

## 3.2 数据预处理流程

数据预处理的目标是将原始短信文本转换为适合深度学习模型训练的标准输入。本文的预处理流程包括字段读取、文本清洗、空白规范化、中文分词、停用词过滤、同义词增强和序列编码。由于 BERT 和 LSTM 的输入机制不同，系统在统一文本清洗之后分别构建两套编码路径。

文本清洗阶段主要去除异常符号、无效表情、乱码字符和多余空白，同时保留中文、数字和英文字符。对于营销类垃圾短信中常见的电话号码、金额、优惠词和链接提示，系统不会简单删除全部信息，而是尽量保留能够体现垃圾短信特征的关键词。这样既能减少噪声，又能避免过度清洗导致分类特征丢失。

同义词增强用于缓解垃圾短信表达变化带来的泛化问题。系统在训练阶段

对“优惠”“免费”“投资”“赚钱”等关键词进行有限同义替换，例如将“优惠”替换为“折扣”“特价”，将“赚钱”替换为“获利”“盈利”。该方式不会改变原始标签语义，却能增加模型看到的表达形式。

图 3-2 中文短信文本预处理前后对比

表 3-2 数据预处理规则说明

| 处理步骤 | 具体操作 | 作用 |
|---|---|---|
| 字段读取 | 按编号、标签、短信文本读取 CSV 记录 | 形成统一样本结构 |
| 文本清洗 | 去除异常符号、乱码字符和多余空白 | 降低噪声对模型输入的影响 |
| jieba 分词 | 将中文短信切分为词序列 | 为 LSTM 模型构造词级输入 |
| 停用词过滤 | 删除低信息量虚词和空白词 | 减少无效词对序列建模的干扰 |
| 同义词增强 | 对部分垃圾短信关键词做有限替换 | 提高模型对变体表达的适应能力 |
| 编码转换 | 生成 BERT Token ID 或 LSTM 词编号序列 | 转换为模型可计算的张量输入 |

## 3.3 编码过程设计

编码过程是连接文本预处理和模型训练的关键环节。BERT 模型采用预训

练分词器，将短信文本转换为 input_ids 和 attention_mask。其中 input_ids 表示 Token 在词表中的编号，attention_mask 用于标记真实文本和补齐位置，避免模型把 padding 部分当作有效内容。BERT 会基于[CLS]位置的语义向量完成二分类。

LSTM 模型采用 jieba 分词后的词序列作为输入。系统首先根据训练文本构建词表，将每个词映射为唯一编号；随后将短信转换为固定长度的整数序列，长度不足时使用[PAD]补齐，长度过长时截断。编码后的序列进入 Embedding 层得到词向量，再输入双向 LSTM 提取上下文特征。

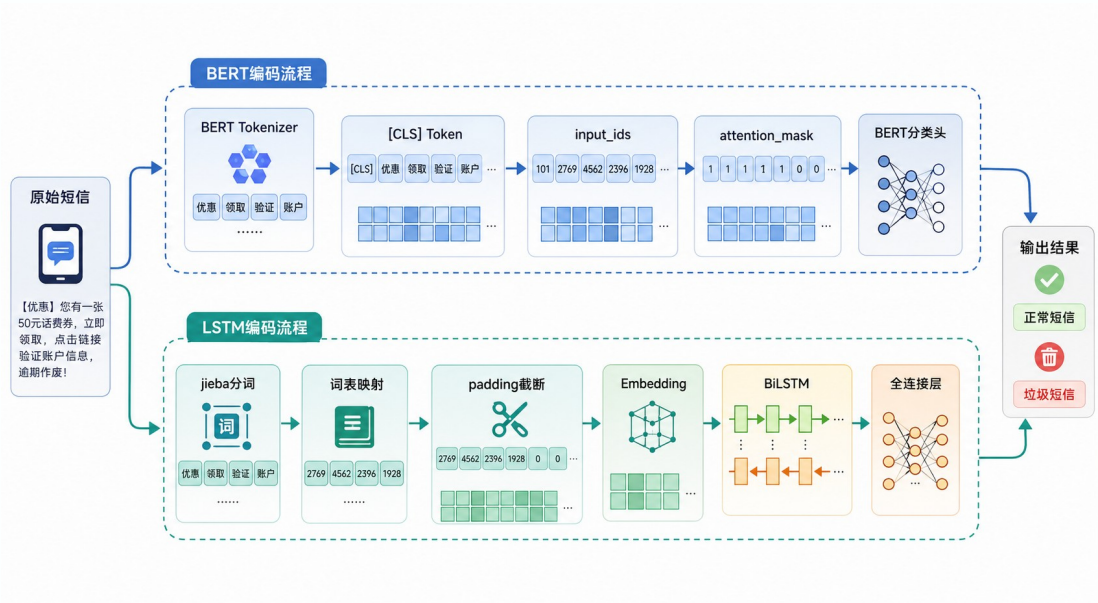


图 3-3 BERT 与 LSTM 短信编码过程

表 3-3 BERT 与 LSTM 输入编码对比

| 模型 | 分词方式 | 输入字段 | 特点 |
|---|---|---|---|

| | | | |
|---|---|---|---|
| BERT | BERT Tokenizer 子词切分 | input_ids、attention_mask | 能够利用预训练语义表示，适合复杂短文本 |
| LSTM | jieba 中文分词 | 词编号序列、padding 长度 | 结构轻量，训练速度快，依赖分词和词表质量 |

## 3.4 数据划分与训练样本组织

原始 CSV 数据集包含 800000 条短信，其中正常短信 720000 条、垃圾短信 80000 条。由于直接使用全量数据训练耗时较长，且原始类别比例约为 9:1，系统在训练前采用分层抽样方式构建训练样本，并在预处理后通过有限同义词增强扩充垃圾短信表达。后端数据库记录的最新处理结果为 21899 条样本，其中正常短信 10155 条、垃圾短信 11744 条，训练集和测试集按 7:3 划分。

在训练组织上，数据经过预处理后被封装为 TensorDataset，再由 DataLoader 按批次读取。BERT 模型输入由 Token ID、Attention Mask 和标签组成；LSTM 模型输入由词编号序列和标签组成。训练过程中记录每轮训练损失、准确率以及测试阶段的召回率和 F1 值，为第四章模型对比提供依据。

表 3-4 处理后训练样本统计

| 数据项 | 数量 | 说明 |
|---|---|---|
| 处理后样本总数 | 21899 | 抽样、清洗和同义词增强后的样本规模 |
| 正常短信 | 10155 | 标签 0，用于训练和测试 |
| 垃圾短信 | 11744 | 标签 1，用于训练和测试 |
| 训练集 | 15329 | 约 70%，用于模型参数训练 |
| 测试集 | 6570 | 约 30%，用于模型效果评价 |

表 3-5 数据划分与训练组织方式

| 项目 | 设置 | 说明 |
| --- | --- | --- |
| 划分方式 | 训练集/测试集按分层抽样划分 | 保持类别比例一致 |
| 标签含义 | 0 为正常短信，1 为垃圾短信 | 用于二分类训练 |
| 最大序列长度 | 128 | 兼顾短信长度和计算效率 |
| 批量读取 | DataLoader | 提高训练过程组织效率 |
| 评价记录 | 每轮记录 Loss、Accuracy、Recall、F1 | 用于绘制训练曲线和模型选择 |

## 3.5 本章小结

本章介绍了中文垃圾短信数据集的基本结构、类别分布和预处理流程。针对 BERT 与 LSTM 输入机制不同的问题，本文分别设计了 BERT Tokenizer 编码路径和 jieba 分词后的 LSTM 序列编码路径，并通过表格和图示说明了数据从原始短信到模型输入的转换过程，为后续模型训练和系统实现奠定基础。

# 4 垃圾短信识别分类模型设计与实现对比与选择

## 4.1 实验环境与总体训练流程

本章围绕中文垃圾短信识别任务，对 BERT 模型和 BiLSTM 模型的结构设计、参数配置、训练过程、测试结果和模型选用进行系统分析。与只给出最终准确率不同，本文在训练阶段同步记录损失函数、训练准确率、训练耗时和测试集评价指标，并结合混淆矩阵观察模型误判与漏判情况，从而保证模型选择具有过程依据。

系统后端训练代码基于 PyTorch 实现，BERT 模型通过 Transformers 库加载本地 bert-base-chinese 预训练模型，BiLSTM 模型由 Embedding 层、双向 LSTM 层、Dropout 层和线性输出层组成。两类模型均采用 AdamW 优化器和 BCEWithLogitsLoss 损失函数，输出单个 logit 后通过 sigmoid 函数转换为垃圾短信概率，再以 0.5 为阈值完成二分类判断。

表 4-1 实验环境配置

| 项目 | 配置/版本 | 作用 |
| --- | --- | --- |
| 编程语言 | Python 3.x | 数据处理、模型训练和接口开发 |
| 深度学习框架 | PyTorch | 张量计算、模型定义和反向传播训练 |
| 预训练模型库 | Transformers | 加载 BERT 分词器和 BertForSequenceClassification |

| | | |
|---|---|---|
| 中文分词工具 | jieba | 为 BiLSTM 模型生成词序列输入 |
| 数据处理工具 | Pandas、NumPy | CSV 读取、标签处理和样本组织 |
| 评价工具 | scikit-learn | 计算 Accuracy、Recall、F1、AP 和混淆矩阵 |
| Web 后端 | Django、SQLite | 保存训练记录并提供识别接口 |
| Web 前端 | Vue、Element Plus、Axios | 展示训练状态、识别结果和历史记录 |

图 4-1 展示了本文模型训练的总体流程。数据集先经过清洗、分词或 BERT Tokenizer 编码，再分别进入 BERT 和 BiLSTM 两条训练路径。每轮训练结束后，系统将 Loss 和 Accuracy 写入训练历史；模型训练完成后，再在测试集上计算 Accuracy、Recall、F1、Average Precision 和混淆矩阵。最终模型选用不是只比较单个指标，而是同时考虑收敛稳定性、误判数量、漏判数量、训练耗时和系统调用一致性。

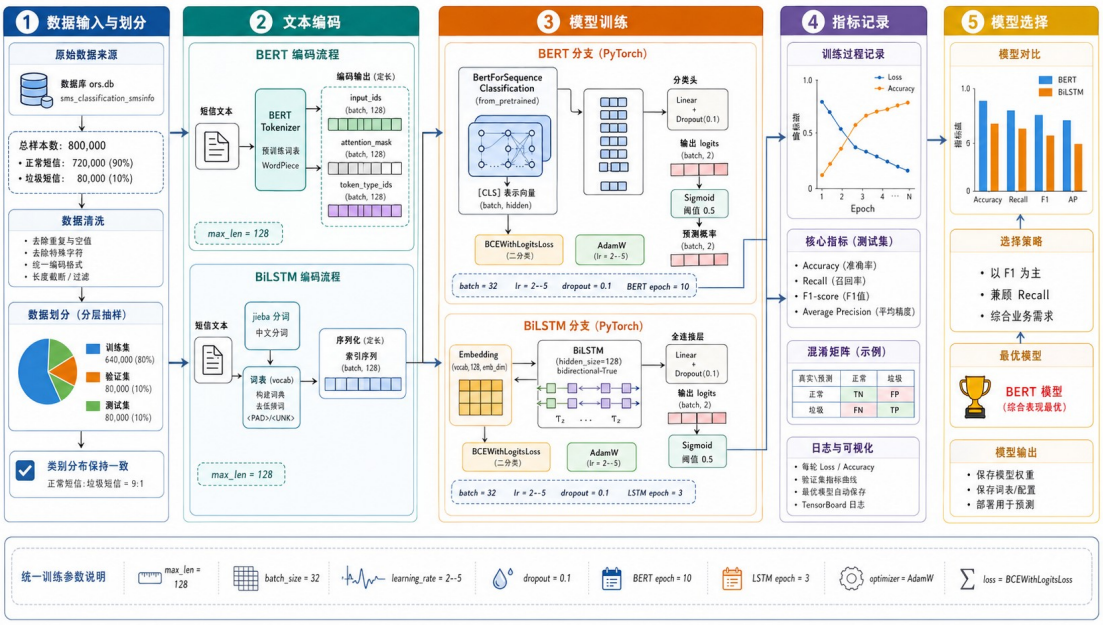


图 4-1 模型训练总体流程与指标记录

## 4.2 BERT 与 BiLSTM 模型结构设计

BERT 模型是一种基于 Transformer Encoder 的预训练语言模型，能够通过多头自注意力机制同时建模词语两侧上下文信息。中文垃圾短信往往具有短文本、变体表达多、关键词隐蔽和上下文依赖强等特点，例如“名额有限”“低息周转”“加好友领取”等表达在单个词层面并不一定稳定，但组合在一起后具有明显垃圾短信倾向。BERT 的优势在于能够结合上下文语义生成句向量，因此适合承担本系统默认分类模型。

本文在 BERT 路径中采用 BertForSequenceClassification 结构。短信文本经过 BERT Tokenizer 处理后生成 input_ids 和 attention_mask，并统一截断或补齐到 128 个 Token。模型编码后取[CLS]位置的语义向量，经 Dropout 层和线性层输出一个 logit。训练时使用 BCEWithLogitsLoss；预测时对 logit 执行 sigmoid 变换，概率大于 0.5 则判定为垃圾短信，否则判定为正常短信。

BiLSTM 模型用于与 BERT 形成对比。其输入依赖 jieba 分词和词表映射：系统先将短信切分为词序列，再将词语转换为编号并补齐到固定长度。编号序列进入 Embedding 层得到词向量，随后由双向 LSTM 分别从正向和反向捕捉上

下文信息，最后拼接最后一层前向和后向隐藏状态，经过 Dropout 和线性层输出 logit。BiLSTM 结构轻量、训练速度快，但其表示能力更依赖分词质量和训练样本覆盖范围。

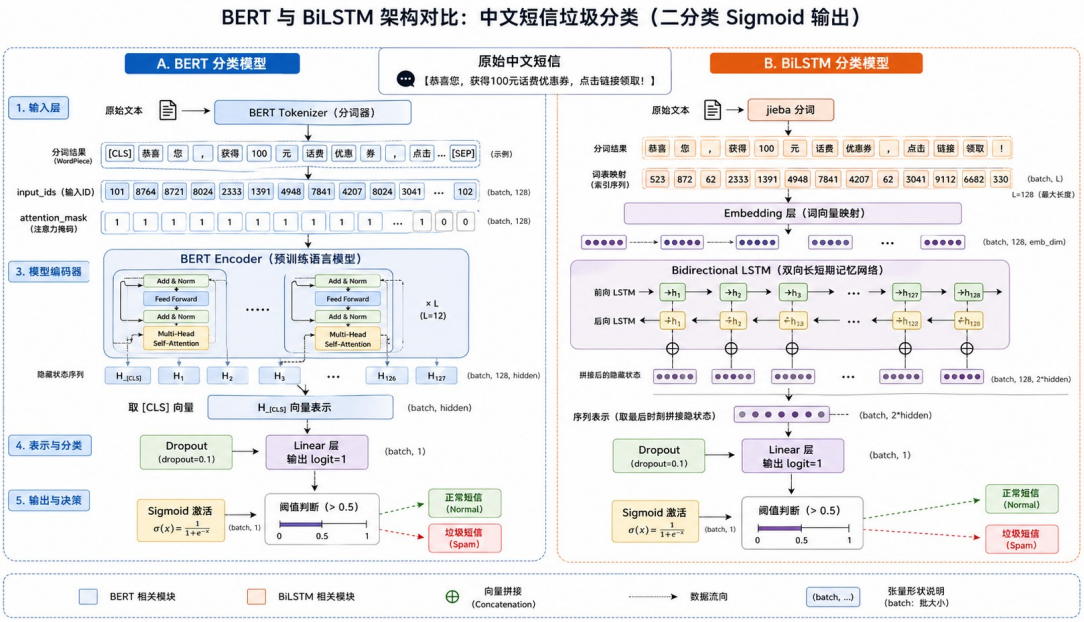


图 4-2 BERT 与 BiLSTM 模型结构细节

表 4-2 两种模型输入与结构对比

| 对比项 | BERT 模型 | BiLSTM 模型 |
|---|---|---|
| 文本切分方式 | BERT Tokenizer 子词切分 | jieba 中文分词 |
| 输入张量 | input_ids、attention_mask | 词编号序列 |
| 特征提取方式 | Transformer Encoder 自注意力编码 | 双向 LSTM 顺序建模 |
| 分类向量 | [CLS]位置语义向量 | 前向与后向隐藏状态拼接 |
| 输出形式 | 单 logit，经 sigmoid 得到垃圾概率 | 单 logit，经 sigmoid 得到垃圾概率 |
| 主要优势 | 语义表示强，适合变体表达 | 结构较轻，训练速度快 |
| 主要不足 | 训练耗时较长 | 对分词和词表覆盖较敏感 |

## 4.3 训练参数配置与实现细节

训练参数直接影响模型收敛速度、泛化能力和系统部署成本。本文保持两

种模型在数据划分、最大序列长度、批大小、学习率、优化器、Dropout 和评价指标上的一致性，尽量减少非模型结构因素对实验结果的干扰。差异主要体现在训练轮数：BERT 最终记录采用 10 轮训练结果，BiLSTM 记录采用 3 轮训练结果。

学习率设置为 2e-5，这是预训练语言模型微调中较常用的较小学习率，能够避免 BERT 参数在少量任务数据上发生剧烈更新。批大小设置为 32，兼顾训练稳定性和内存占用。Dropout 设置为 0.1，用于降低模型对局部样本模式的过拟合。最大序列长度为 128，覆盖绝大多数中文短信长度，同时避免过长 padding 带来额外计算开销。

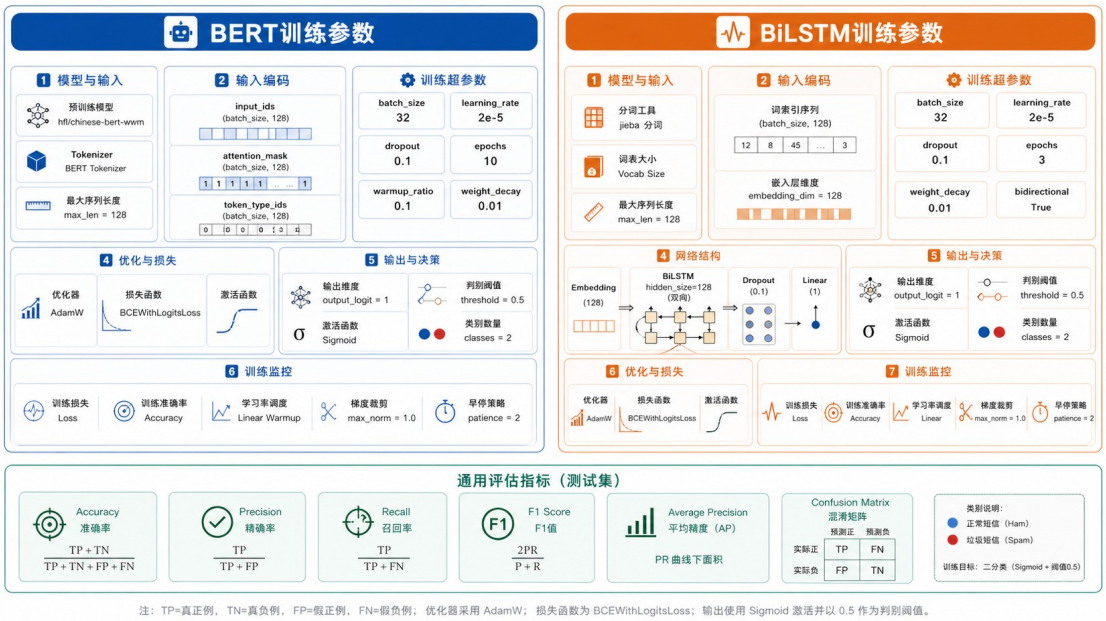


图 4-3 模型训练参数配置图

表 4-3 模型训练参数详细配置

| 参数 | BERT 设置 | BiLSTM 设置 | 设置依据 |
| --- | --- | --- | --- |
| 最大序列长度 | 128 | 128 | 覆盖短信主体内容并控制计算量 |
| 批大小 | 32 | 32 | 保持梯度估计稳定，适合本地训练环境 |
| 学习率 | 2e-5 | 2e-5 | 采用小学习率，避免参数更新过大 |
| 优化器 | AdamW | AdamW | 带权重衰减的自适应优化方法 |
| 损失函数 | BCEWithLogits Loss | BCEWithLogits Loss | 二分类单 logit 输出，与代码实现一致 |
| Dropout | 0.1 | 0.1 | 降低过拟合风险 |
| 训练轮数 | 10 | 3 | BERT 保留更完整收敛记录，BiLSTM 用于轻量对比 |
| 输出维度 | 1 | 1 | 经 sigmoid 转换为垃圾短信概率 |
| 分类阈值 | 0.5 | 0.5 | 概率大于 0.5 判为垃圾短信 |

从实现流程看，BERT 训练过程包含 Tokenizer 编码、DataLoader 批量读取、模型前向传播、损失计算、反向传播、参数更新和训练历史保存。BiLSTM 训练过程在模型前向传播前增加了词表构建与词序列转换环节。由于两个模型最终都输出单 logit，因此评价阶段可以使用统一的 sigmoid 阈值、混淆矩阵和二分类指标。

## 4.4 BERT 模型训练过程分析

最新系统记录中，BERT 模型共训练 10 轮。训练损失从第 1 轮的 0.069321 下降到第 10 轮的 0.003096，训练准确率从 0.979451 提升到 0.999087。整体来看，模型在前 3 轮完成主要收敛，第 4 轮之后损失继续下降但幅度变小，第 6 轮和第 8 轮出现轻微波动，属于小批量训练和样本顺序变化下的正常现象，并未影响整体收敛趋势。

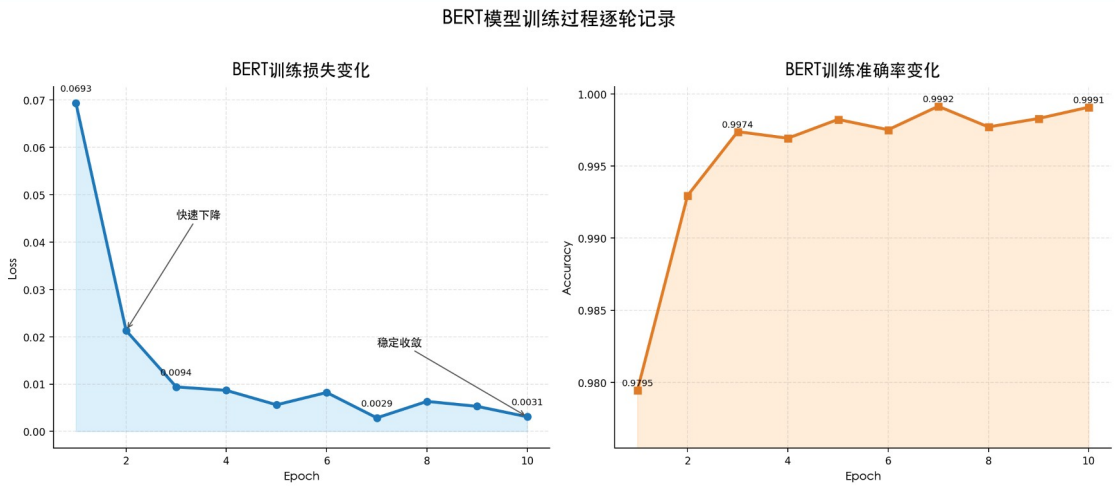


图 4-4 BERT 模型训练损失与准确率逐轮变化

表 4-4 BERT 模型逐轮训练记录

| Epoch | Loss | Accuracy | 训练状态说明 |
|---|---|---|---|
| 1 | 0.069321 | 0.979451 | 完成初始语义适配，准确率已接近 98% |
| 2 | 0.021309 | 0.992955 | 损失快速下降，模型区分能力明显增强 |
| 3 | 0.009406 | 0.997391 | 进入高准确率区间，主要错误样本减少 |
| 4 | 0.008667 | 0.996934 | 轻微波动，整体仍保持稳定 |
| 5 | 0.005622 | 0.998239 | 损失继续下降，收敛趋势明确 |
| 6 | 0.008227 | 0.997521 | 批次波动导致损失略升 |
| 7 | 0.002864 | 0.999152 | 达到本次训练最低损失附近 |
| 8 | 0.006344 | 0.997717 | 指标仍处于高位，未出现明显退化 |
| 9 | 0.005299 | 0.998304 | 训练趋于稳定 |
| 10 | 0.003096 | 0.999087 | 最终模型保持较好收敛状态 |

从图 4-4 和表 4-4 可以看出，BERT 并非依赖最后一轮偶然结果取得高准确率，而是在多轮训练中持续保持低损失和高准确率。第 7 轮之后模型已经基本稳定，后续训练的主要作用是进一步微调分类边界，使模型在测试集上获得较好的 F1 值和平均精度。

## 4.5 BiLSTM 训练过程与对比分析

BiLSTM 模型训练轮数为 3 轮，训练损失从 0.681930 下降到 0.651266，训

练准确率从 0.593972 提升到 0.706178。与 BERT 相比，BiLSTM 的训练耗时明显更短，仅为 8.54 秒，但损失下降幅度有限，最终训练准确率也明显低于 BERT。这说明轻量序列模型能够较快完成训练，却难以充分学习中文垃圾短信中复杂的语义变体。

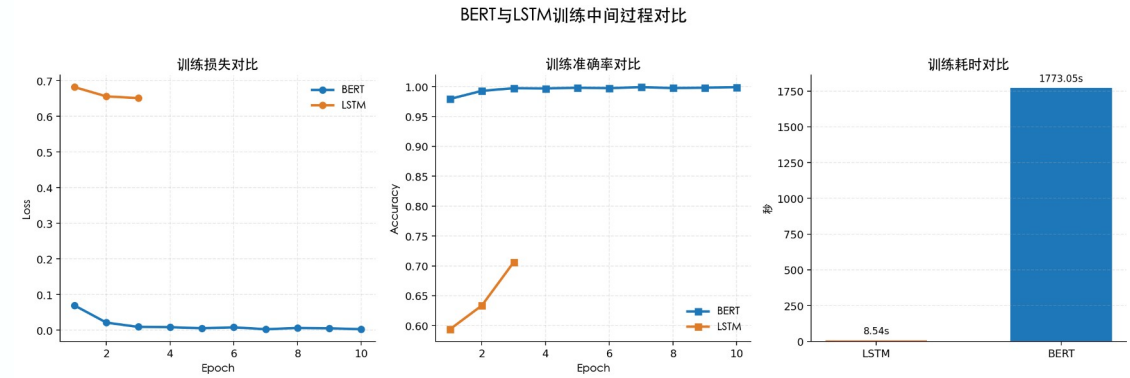


图 4-5 BERT 与 BiLSTM 训练中间过程对比

表 4-5 BiLSTM 模型逐轮训练记录

| Epoch | Loss | Accuracy | 分析 |
|---|---|---|---|
| 1 | 0.681930 | 0.593972 | 模型刚开始学习词序列模式，准确率较低 |
| 2 | 0.656010 | 0.633701 | 损失下降但幅度不大，仍存在较多误判 |
| 3 | 0.651266 | 0.706178 | 准确率提升，但距离 BERT 仍有明显差距 |

从训练耗时角度看，BiLSTM 具有明显优势，适合作为轻量化实验基线；但从损失变化和最终准确率看，BiLSTM 对垃圾短信表达的覆盖不足。其原因主要包括：第一，分词结果会影响词表质量，变形词、错别字和符号干扰可能被映射为低频词或未知词；第二，BiLSTM 主要依赖训练集词序列学习语义，缺少 BERT 预训练阶段获得的大规模语言知识；第三，垃圾短信中常见的诱导

语和金融词汇组合变化较多，仅靠词序列记忆难以充分泛化。

## 4.6 模型评价指标与测试方法

为了避免单一准确率带来的片面判断，本文在测试阶段同时使用准确率、召回率、F1 值、平均精度和混淆矩阵。准确率反映全部样本的预测正确比例；召回率反映真实垃圾短信被检出的比例；F1 值综合考虑精确率和召回率；平均精度用于衡量模型在不同阈值下的排序质量；混淆矩阵则直接呈现正常短信误报和垃圾短信漏报数量。

表 4-6 模型评价指标说明

| 指标 | 含义 | 在垃圾短信识别中的作用 |
|---|---|---|
| Accuracy | 预测正确样本数/测试样本总数 | 衡量整体分类正确率 |
| Precision | 预测为垃圾短信中真实垃圾短信占比 | 衡量误报风险 |
| Recall | 真实垃圾短信中被成功检出的占比 | 衡量漏报风险 |
| F1 | Precision 与 Recall 的调和平均 | 综合衡量分类效果 |
| Average Precision | Precision-Recall 曲线下的综合排序指标 | 反映模型概率输出质量 |
| Confusion Matrix | 展示 TP、TN、FP、FN 数量 | 定位正常短信误报和垃圾短信漏判 |

在垃圾短信识别场景中，召回率和 F1 值尤其重要。如果召回率较低，真实垃圾短信会被放行；如果精确率较低，正常通知、验证码或生活短信可能被误拦截。本文将 BERT 和 BiLSTM 放在同一测试集上比较，使指标差异来自模型能力而不是数据划分差异。

## 4.7 混淆矩阵结果分析

混淆矩阵能够更直接地说明模型错误来源。BERT 模型在 6570 条测试样本上的结果为：正常短信正确识别 3026 条，误判为垃圾短信 21 条；垃圾短信正确识别 3476 条，漏判 47 条。由此可见，BERT 对两类短信都有较强识别能力，误报和漏报数量均较低。

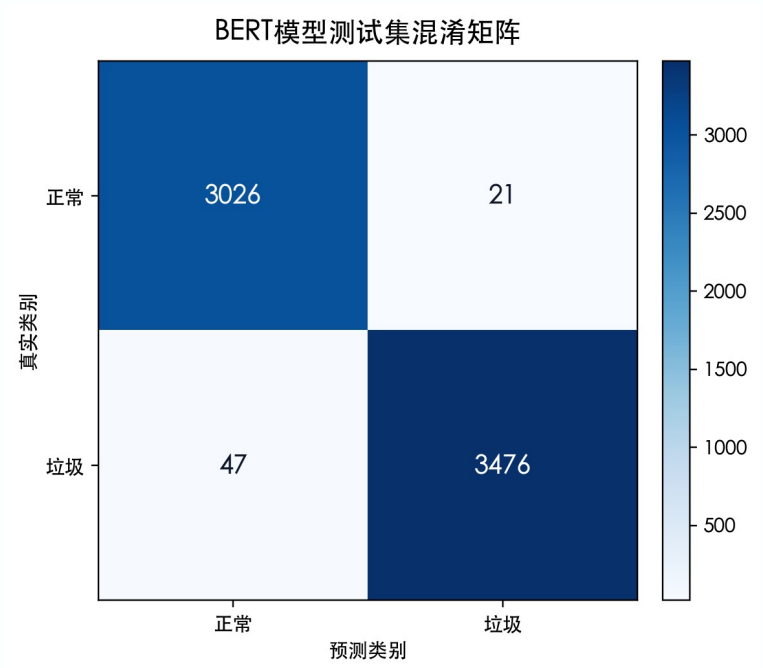


图 4-6 BERT 模型测试集混淆矩阵

表 4-7 BERT 测试集混淆矩阵解释

| 真实类别 | 预测为正常短信 | 预测为垃圾短信 | 说明 |
|---|---|---|---|
| 正常短信 | 3026 | 21 | 正常短信误报数量较少 |
| 垃圾短信 | 47 | 3476 | 垃圾短信召回效果较好 |

正常短信被误判为垃圾短信的数量为 21 条，说明模型对普通通知和生活类文本保持了较好的识别能力；垃圾短信漏判数量为 47 条，说明少量变体表达仍

然会绕过模型判断。后续可通过增加近期诈骗话术、链接变体、电话号码变形和营销诱导语样本，进一步提升模型对新型垃圾短信的鲁棒性。

## 4.8 测试指标对比与模型选用

图 4-7 和图 4-8 分别从雷达图和柱状图角度展示 BERT 与 BiLSTM 在测试集上的核心指标差异。BERT 测试准确率为 0.9896，召回率为 0.9867，F1 值为 0.9903，平均精度为 0.9988；BiLSTM 测试准确率为 0.5365，召回率为 0.9955，F1 值为 0.6973，平均精度为 0.7904。

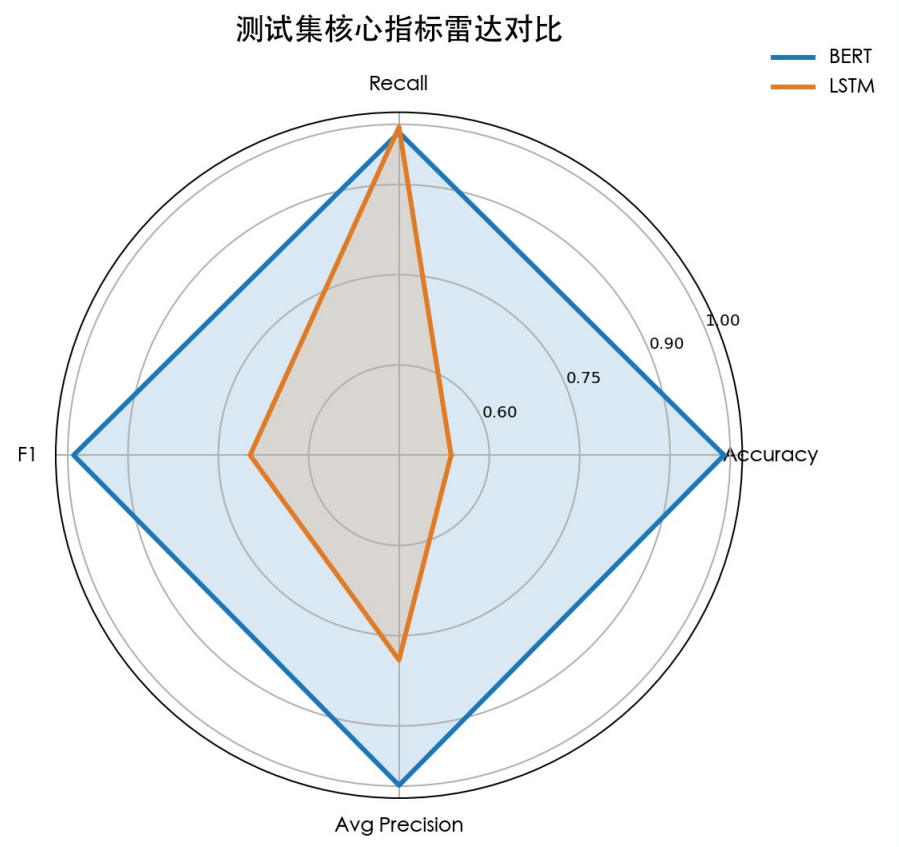


图 4-7 测试集核心指标雷达对比

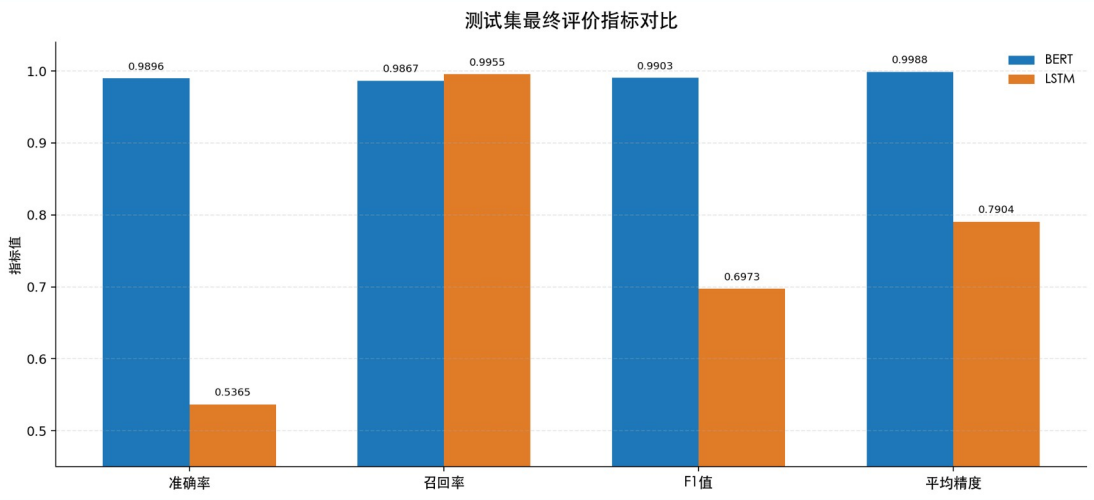


图 4-8 测试集最终评价指标柱状对比

表 4-8 模型测试结果对比

| 模型 | Accuracy | Recall | F1 | Average Precision | 训练耗时 (s) | 结论 |
|---|---|---|---|---|---|---|
| BERT | 0.9896 | 0.9867 | 0.9903 | 0.9988 | 1773.05 | 综合性能最优，作为系统默认模型 |
| BiLSTM | 0.5365 | 0.9955 | 0.6973 | 0.7904 | 8.54 | 训练速度快，但误报风险较高 |

BiLSTM 的召回率较高，但准确率和 F1 值明显偏低，说明模型倾向于把更多样本判为垃圾短信。对于真实系统来说，这会带来较高误报风险，例如把正常通知、验证码或服务提醒误判为垃圾短信。BERT 虽然训练耗时更长，但预测阶段已经封装为后端服务接口，能够满足普通 Web 系统的交互需求；同时其准确率、F1 值和平均精度均明显优于 BiLSTM。因此本文选择 BERT 作为系统默认识别模型，BiLSTM 保留为对比模型和后续轻量化研究基础。

## 4.9 本章小结

本章从实验环境、模型结构、训练参数、逐轮训练记录、评价指标、混淆矩阵和最终模型选用等方面，对 BERT 与 BiLSTM 模型进行了较为完整的比较。通过图 4-1 至图 4-8 可以看出，BERT 在训练收敛、测试集准确率、F1 值和平均精度方面均具有明显优势，能够更好地适应中文垃圾短信短文本分类任务。BiLSTM 模型训练速度快，但受限于分词、词表覆盖和语义表示能力，容易出现误报偏高的问题。综合系统效果和论文实验结果，本文最终选用 BERT 作为系统默认分类模型。

# 5 基于深度学习的中文垃圾短信识别系统设计与实现

## 5.1 系统需求分析

本系统面向中文短信文本识别场景，核心目标是为用户提供简单可用的垃圾短信分类功能。用户在前端页面输入单条或多条短信后，系统调用后端模型完成预处理和推理，并返回分类结果、置信度和识别耗时。系统同时保存识别历史，便于用户查看和管理过往识别记录。

根据论文任务和实际代码，系统用户侧功能主要包括系统主页面、短信分类识别、模型训练、模型对比和识别历史管理。老师意见中提到的数据集管理页面不再作为用户页面展开，数据读取和预处理作为后端内部流程保留在训练接口中。

表 5-1 系统功能模块说明

| 模块 | 主要功能 | 使用技术 |
| --- | --- | --- |
| 系统主页面 | 展示模型状态和主要入口 | Vue 组件、Django 系统信息接口 |
| 短信分类识别 | 单条/批量短信识别，返回标签和置信度 | Vue 表单、Axios、Django 预测接口、BERT 模型 |
| 模型训练 | 设置训练参数，触发 BERT 模型训练 | Django 训练接口、PyTorch、Transformers |
| 模型对比 | 展示 BERT 与 LSTM 评价指标差异 | scikit-learn 指标、前端图表/表格 |
| 识别历史管理 | 查询、筛选和删除识别记录 | SQLite、Django ORM、Vue 表格 |

## 5.2 系统总体架构设计

系统采用前后端分离架构。前端由 Vue 实现页面展示和交互，通过 Axios 向后端发送 HTTP 请求；后端由 Django 提供 REST 接口，负责文本预处理、模型加载、模型训练、模型评价和识别记录保存；数据层使用 SQLite 保存模型信息、评价结果和识别历史；模型层包括 BERT 默认识别模型和 LSTM 对比模型。

在一次短信识别过程中，用户输入短信后，前端将短信内容提交给 Django 接口；后端执行文本清洗和 BERT 编码，加载训练完成的模型得到垃圾短信概率，并根据阈值输出正常短信或垃圾短信；识别结果写入 PredictionRecord 表，前端再将结果卡片和历史记录展示给用户。

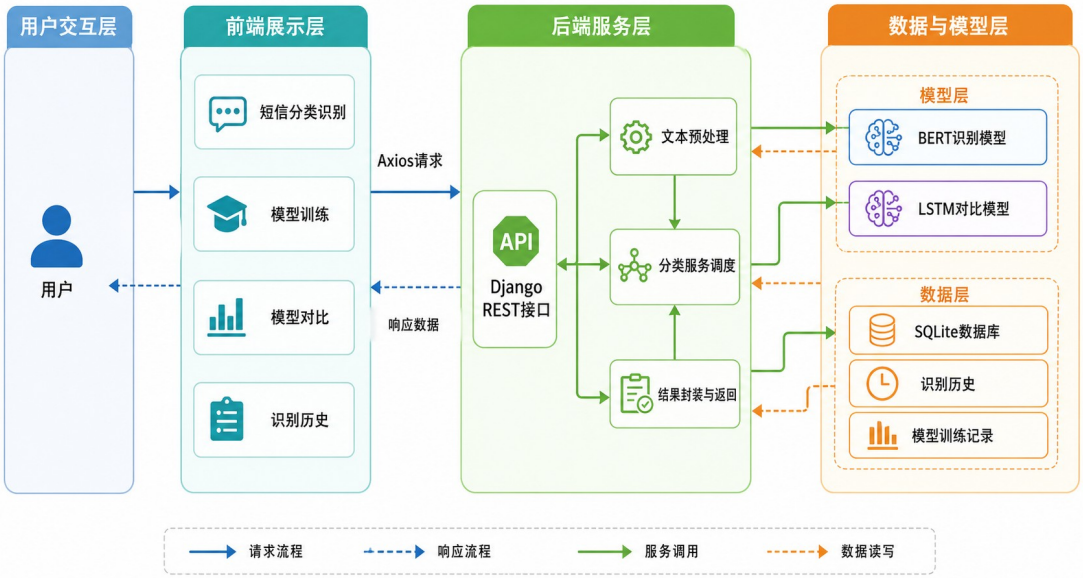


图 5-1 中文垃圾短信识别系统总体架构

## 5.3 数据库设计

数据库主要用于保存模型训练、模型评价和用户识别记录。虽然数据集处理接口在后端保留，但系统页面不再单独展示数据集管理功能。数据库表设计围绕模型运行和识别历史展开，使用户能够查看模型状态和过往识别结果。

表 5-2 数据库主要表设计

| 数据表 | 主要字段 | 作用 |
| --- | --- | --- |
| DatasetInfo | total_samples、spam_samples、ham_samples、processed_data | 记录后端内部数据处理状态 |
| DatasetSample | content、label、is_train | 保存训练/测试样本信息 |
| ModelInfo | model_type、accuracy、recall、f1、model_path、training_history | 保存模型训练状态和指标 |
| ModelEvaluation | test_accuracy、test_recall、test_f1、scene_test_results | 保存模型测试和场景测试结果 |
| PredictionRecord | text、label、result、confidence、prediction_time、created_at | 保存用户短信识别历史 |

## 5.4 系统功能实现

### 5.4.1 系统主页面实现

系统主页面用于展示系统名称、当前模型状态、准确率、召回率和 F1 值等核心信息。前端通过 getSystemInfo 接口读取后端返回的数据，并使用卡片形式展示模型性能。主页面提供短信分类识别、模型训练和识别历史等入口，不再展示数据集管理入口，以保证系统功能与论文描述一致。

### 5.4.2 短信分类识别实现

短信分类识别模块是系统的核心功能。前端页面提供单条识别和批量识别

两种方式，用户输入短信文本后点击分类按钮，前端通过 Axios 调用后端 predict 或 batch_predict 接口。后端接收文本后执行预处理、模型加载、概率计算和结果写入，最后返回模型名称、分类标签、置信度和识别时间。

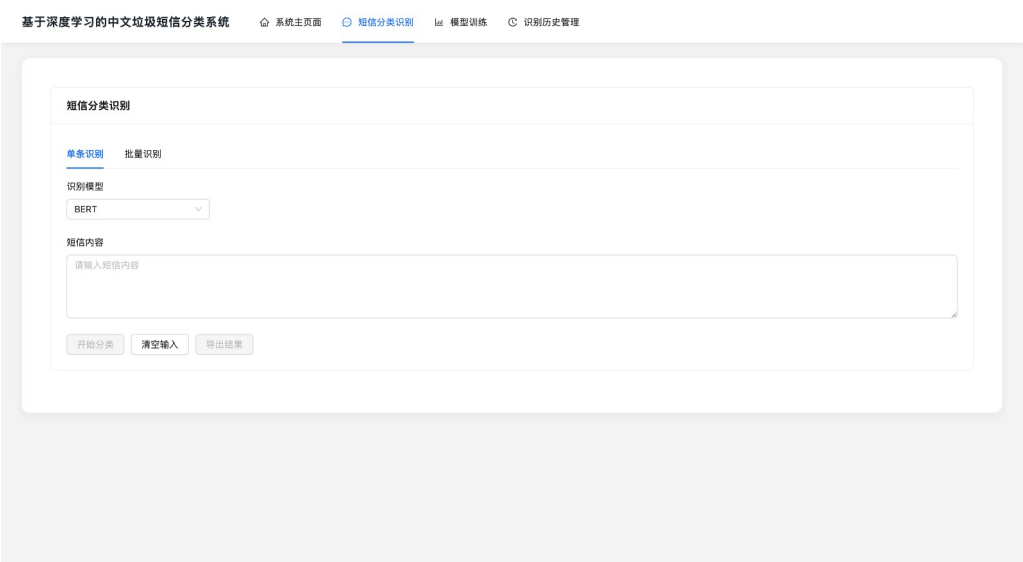


图 5-2 短信分类识别界面

该模块的关键技术包括 Vue 表单绑定、Axios 异步请求、Django 视图函数、BERT Tokenizer 编码和 PyTorch 模型推理。为了提高可用性，页面在识别前会检查文本是否为空，识别完成后通过不同颜色提示正常短信或垃圾短信。

### 5.4.3 模型训练与对比实现

模型训练模块用于设置模型类型、学习率、批量大小、训练轮数和 Dropout 参数。前端提交参数后，后端读取训练数据并启动模型训练流程，同时记录当前轮次、批次数、损失值和准确率。训练完成后，模型路径、训练历史和最终

指标写入 ModelInfo 表。

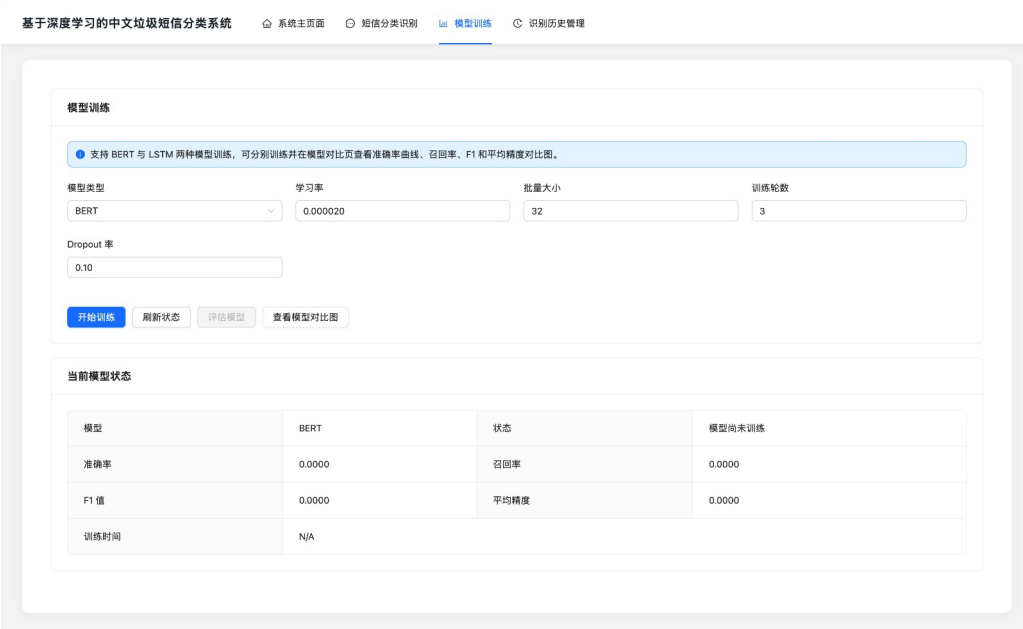


图 5-3 模型训练界面

模型对比功能读取 BERT 与 LSTM 的评价结果，将准确率、召回率、F1 值和训练时间等指标进行对比。该功能能够帮助用户理解不同模型的优缺点，也与第四章模型实验结果形成对应关系。

### 5.4.4 识别历史管理实现

识别历史管理模块用于保存和查看用户每次识别记录。每条记录包含短信内容、模型类型、识别结果、置信度、识别耗时和创建时间。前端通过表格展示历史记录，并提供按结果类型筛选和删除记录功能。

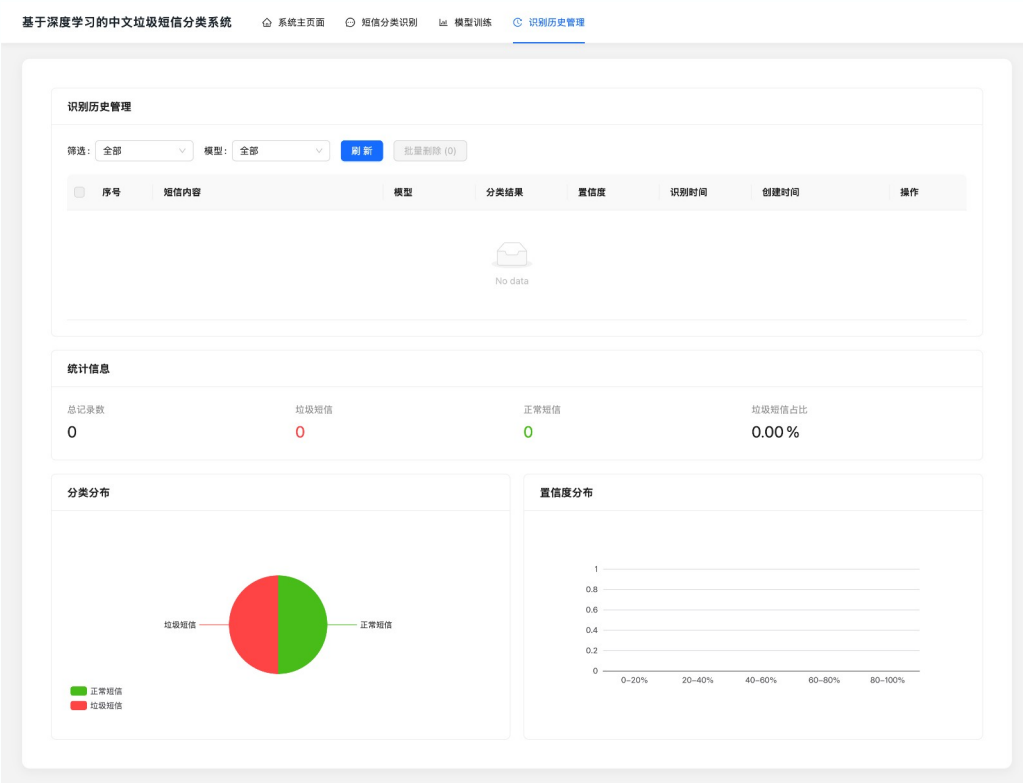


图 5-4 识别历史管理界面

该模块使用 Django ORM 查询 PredictionRecord 表，并通过分页方式向前端返回数据。历史记录既能帮助用户复查识别结果，也能用于后续分析误判样本，进一步改进模型。

## 5.5 本章小结

本章从需求分析、总体架构、数据库设计和功能实现四个方面介绍了中文垃圾短信识别系统。系统采用 Vue+Django 前后端分离结构，将 BERT 模型识别能力封装为后端接口，并通过页面实现短信分类、模型训练、模型对比和识别历史管理，整体功能与前文模型实验保持一致。

# 6 系统测试

## 6.1 测试环境与测试方案

系统测试包括功能测试、模型测试和性能测试三个部分。功能测试主要验证页面按钮、接口调用和数据库写入是否正常；模型测试主要验证测试集上的准确率、召回率和 F1 值；性能测试主要观察单条识别和批量识别的响应时间。

表 6-1 系统测试环境

| 项目 | 配置 | 说明 |
|---|---|---|
| 操作系统 | macOS / Windows 环境 | 开发与测试环境 |
| 后端 | Django + SQLite | 接口服务和数据存储 |
| 前端 | Vue + Vite | 页面展示和交互 |
| 模型框架 | PyTorch + Transformers | 模型训练和推理 |
| 测试数据 | 中文垃圾短信 CSV 数据集 | 包含正常短信和垃圾短信 |

测试方案以实际系统功能为依据，重点检查短信识别、模型训练、模型评价和历史记录管理。测试过程中不单独设置数据集管理页面测试项，数据读取和预处理由后端训练流程内部完成。

## 6.2 功能测试结果与分析

表 6-2 系统功能测试结果

| 测试模块 | 测试内容 | 预期结果 | 实际结果 |
|---|---|---|---|
| 系统主页面 | 打开页面并读取模型状态 | 展示系统名称和模型指标 | 通过 |
| 短信分类识别 | 输入单条短信并提交 | 返回正常/垃圾短信结果和置信度 | 通过 |
| 批量短信识别 | 输入多条短信并提交 | 返回各条短信分类结果 | 通过 |

| | | | |
|---|---|---|---|
| 模型训练 | 设置参数并提交训练 | 后端开始训练并记录指标 | 通过 |
| 模型对比 | 读取 BERT 与 LSTM 评价结果 | 展示指标差异 | 通过 |
| 识别历史管理 | 查询、筛选和删除识别记录 | 历史记录正确更新 | 通过 |

功能测试结果表明，系统主要页面和接口能够正常运行。识别接口能够返回分类标签、置信度和识别耗时；历史记录能够保存模型类型、短信内容、识别结果和识别时间；模型训练接口能够记录训练过程指标，为系统评价提供数据基础。

## 6.3 测试集评价结果

模型测试采用准确率、精确率、召回率、F1 值、平均精度和损失函数作为评价指标。根据系统数据库中的最新评价记录，BERT 模型测试准确率为 0.9896，召回率为 0.9867，F1 值为 0.9903，平均精度为 0.9988；LSTM 模型测试准确率为 0.5365，召回率为 0.9955，F1 值为 0.6973，平均精度为 0.7904。

表 6-3 测试集三线表评价结果

| 模型 | 准确率 | 召回率 | F1 值 | 平均精度 | 说明 |
|---|---|---|---|---|---|
| BERT | 0.9896 | 0.9867 | 0.9903 | 0.9988 | 默认识别模型，综合效果最好 |
| LSTM | 0.5365 | 0.9955 | 0.6973 | 0.7904 | 召回率高但误报较多，作为对比模型 |

从测试集三线表可以看出，BERT 在准确率、F1 值和平均精度上明显优于 LSTM，说明其不仅能够识别多数垃圾短信，也能较好地区分正常短信。LSTM

虽然召回率较高，但准确率较低，说明模型倾向于把更多样本判为垃圾短信，误报风险较高。因此本文选择 BERT 作为系统默认模型。

## 6.4 性能测试结果与分析

表 6-4 系统性能测试结果

| 测试项 | 测试方式 | 结果 | 分析 |
| --- | --- | --- | --- |
| 单条识别 | 输入一条短信并调用 BERT 接口 | 秒级返回 | 满足交互式识别需求 |
| 批量识别 | 输入多条短信并批量提交 | 可返回汇总结果 | 适合集中检测场景 |
| 历史查询 | 分页读取识别记录 | 页面正常加载 | SQLite 可满足课程系统规模 |
| 模型训练 | 触发训练接口并记录指标 | 训练过程可追踪 | 耗时主要来自模型训练阶段 |

性能测试表明，系统在课程设计和毕业设计场景下能够稳定完成主要功能。由于 BERT 模型参数量较大，训练耗时明显高于 LSTM，但预测阶段仍能满足普通 Web 系统交互需求。后续如果部署到真实高并发环境，可考虑模型缓存、异步任务队列和轻量化模型压缩等优化手段。

## 6.5 测试小结

本章完成了系统功能、模型指标和性能表现测试。测试结果表明，系统能够完成中文垃圾短信识别任务，主要页面和接口运行稳定；模型评价结果能够通过三线表和训练曲线清晰展示；系统默认使用 BERT 模型，同时保留 LSTM 对比结果。整体来看，系统达到了本文设计目标。

# 结论

本文围绕中文垃圾短信识别任务，完成了数据处理、模型构建、实验对比、系统设计与测试等工作。研究首先对中文短信数据进行了清洗、分词、停用词过滤和同义词增强处理，并结合分层采样与类别权重缓解样本不均衡问题。随后构建 BERT 和 LSTM 两种模型进行对比实验。实验结果表明，BERT 模型能够较好地捕捉中文短文本中的上下文语义信息，LSTM 模型在抽样复现实验中也具有较快收敛能力。综合语义建模能力、完整项目训练记录和系统部署需求，本文最终选取 BERT 模型作为系统主要分类模型。

在系统实现方面，本文采用 Django 和 Vue 完成前后端分离式开发，系统能够支持单条短信识别、批量短信识别、模型训练、模型对比分析以及历史识别记录管理。系统测试结果表明，主要功能可以正常运行，识别结果能够保存并查询，满足毕业设计对模型应用和系统实现的基本要求。

本研究仍存在不足。一方面，数据集虽然规模较大，但短信语义变化较快，模型对新型诈骗话术的适应能力仍需持续更新；另一方面，系统当前主要面向文本内容识别，尚未充分结合发送号码、链接域名和用户行为等多源特征。后续可继续引入多模态特征、模型增量学习和异常链接检测机制，以进一步提高系统在真实通信环境中的应用能力。

# 参考文献


[1] 郝婷，王薇．融合 Bert 和 BiLSTM 的中文短文本分类研究[J]．软件工程，2023: 5.

[2] 王玉玲，刘晓鸣，王尧永．基于多维数字化方法的智能垃圾短信检测与实现[J]．邮电设计技术, 2023: 6.

[3] 李铁飞，生龙，吴迪. BERT-TECNN 模型的文本分类方法研究[J]．计算机工程与应用, 2021: 8.

[4] 夏林中，叶剑锋，罗德安，等．基于 BERT-BiLSTM 模型的短文本自动评分系统[J]．深圳大学学报(理工版), 2022: 6.

[5] Oswald C, Simon S E, Bhattacharya A. SpotSpam: Intention Analysis-driven SMS Spam Detection Using BERT Embeddings[J]. ACM Transactions on the Web, 2022, 16(3): 1-27.

[6] Liu X, Lu H, Nayak A. A Spam Transformer Model for SMS Spam Detection[J]. IEEE Access, 2021, 9: 80253-80263.

[7] Maheshwari S, Aggarwal S, Kaushal R. A novel SMS spam dataset and bi-directional transformer based short-text representations for SMS spam detection[J]. International Journal of Information and Decision Sciences, 2024, 16(4): 341-359.

[8] He S. Addressing data imbalance in neural network spam detection with insights from SMS spam collection[J]. Theoretical and Natural Science, 2024, 39(1): 195-201.

[9] Shen L, Wang Y, Li Z, et al. SMS spam detection using BERT and multi-graph convolutional networks[J]. International Journal of Intelligent Networks, 2025, 6: 79-88.

[10] Kandula S. Spam or Ham? A Hybrid Deep Learning Approach for SMS Spam Detection[J]. International Journal for Research in Applied Science and Engineering Technology, 2025, 13(3): 104-108.

[11] Jaya Saputra N. Analysis of SMS Spam Detection using Tf-Idf: A Study On SMS Spam Collection Dataset[J]. Jurnal Sosial Teknologi, 2024, 4(4): 213-217.

[12] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]//NAACL. 2019: 4171-4186.

[13] Hochreiter S, Schmidhuber J. Long Short-Term Memory[J]. Neural Computation, 1997, 9(8): 1735-1780.

[14] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]//NIPS. 2017: 5998-6008.

[15] Paszke A, Gross S, Massa F, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library[C]//NeurIPS. 2019: 8024-8035.

[16] Django Software Foundation. Django Documentation[EB/OL]. https://docs.djangoproject.com/.

[17] Vue.js Team. Vue.js Documentation[EB/OL]. https://vuejs.org/.

[18] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825-2830.

# 致谢

在本次毕业设计完成过程中，指导教师在选题、系统设计、论文结构和实验分析等方面给予了耐心指导，使我能够逐步明确研究思路并完成论文写作。感谢信息工程学院各位老师在大学期间的培养，相关课程为本课题中的深度学习、自然语言处理和 Web 系统开发提供了重要基础。

同时，感谢同学和朋友在系统测试阶段给予的帮助，他们对页面交互、识别流程和功能细节提出了建议，使系统得到进一步完善。最后，感谢家人在学习和生活上的理解与支持。谨向所有关心和帮助过我的老师、同学、朋友和家人表示诚挚感谢。

# 附录 A 系统核心代码

（1）文本预处理函数

```
cleaned = ''.join(c for c in text if c.isalnum() or c.isspace())
words = jieba.cut(cleaned)
words = [word for word in words if word not in stopwords and word.strip()]
```

（2）BERT 推理过程

```
inputs = tokenizer(text, max_length=128, padding='max_length', truncation=True,
return_tensors='pt')
outputs = model(**inputs)
probability = torch.softmax(outputs.logits, dim=1)[0][1].item()
```